

CS120 Project Report

Sen LI

lisen2022@shanghaitech.edu.cn

Yourong Cao

caoyr2022@shanghaitech.edu.cn

Abstract

In the CS120 project, we utilized tools such as JUCE, ASIO, and Npcap, and developed an Ethernet based on audio signal transmission using both C++ and Python programming languages on development platforms VS and VScode. We also accomplished the following task points.

- Project 0. Warm up
 - Task 1: (0/0 point) Self-assessment
 - Task 2: (3/3 points) Understanding Your Tools
- Project 1. Acoustic Link
 - Task 0: (0/0 points) Audio Toolkit
 - Task 1: (3/3 points) Understanding Your Tools
 - Task 2: (2/2 points) Generating Sound Waves at Will
 - Task 3: (0/5 points) Transmitting Your First Bit
 - Task 4: (0/1 point) Error Correction
 - Task 5: (0/2 points) OFDM
 - Task 6: (0/2 points) Chip Dream
 - Task 7: (0/1 point) MIMO
 - Task 8: (0/2 points) Range Challenge
- Project 2. Manage Multiple Access
 - Task 1: (0/4 points) Cable Connection
 - Task 2: (0/5 points) Acknowledgement
 - Task 3: (0/2 points) Carrier Sense Multiple Access
 - Task 4: (0/1 point) CSMA with Interference
 - Task 5: (0/3 points) Performance Rank
 - Task 6: (0/3 points) X
- Project 3. To the Internet
 - Task 0: (0/0 point) Sending and Receiving IP Datagram

- Task 1: (2.4/3 points) ICMP Echo
 - Task 2: (4/4 points) Router
 - Task 3: (0/3 points) NAT
 - Task 4: (1/1 point) NAT Traversal
 - Task 5: (1/1 point) IP Fragmentation
 - Task 6: (0/0 point) Virtual Network Device
 - Task 7: (1/1 point) ICMP Echo #
 - Task 8: (1/1 point) Router #
 - Task 9: (0/1 point) NAT #
 - Task 10: (1/1 point) ARP
 - Task 11: (0/* points) Star
- Project 4. Above IP
 - Task 1: (4/4 points) DNS
 - Task 2: (1/1 point) HTTP
 - Task 3: (3/3 points) Project Report
 - Task 4: (0/0 point) Relaunch
 - Task 5: (1/1 point) Browsing the Web
 - Task 6: (1/1 point) Project Report #

1. Project 0. Warm Up

1.1. Stories

At the beginning, when we encountered JUCE, we chose to follow the tutorials provided by them for learning. We set up the environment on VS and learned some of JUCE's modules and necessary functions. We also successfully replicated the sample of time-delay signal.

1.2. Designs

We chose to implement the callback functions of the JUCE module ourselves and compiled and ran the program on VS. We then conducted a test for the output of the sound signal.

1.3. Challenges

JUCE has a large number of modules and standard functions, and the course project also requires the use of ASIO driver. When starting to use it, on the one hand, due to hardware compatibility issues (for example, the onboard driver of ASUS is incompatible with ASIO4ALL, causing JUCE to be unable to detect the device after starting the driver), one may encounter difficulties when configuring the environment. On the other hand, when completing the project, the lack of understanding of the function of the functions led to a lack of direction at the beginning.

1.4. Solutions

Combining the project experience of others and AI, first understand the key framework that can be used to build the project, and then when completing the project, learn the relevant functions as needed.

1.5. Suggestions

Before the project begins, it is possible to arrange one or two tutorial sessions to introduce the main framework of JUCE. For a sample of delayed signal output and the things needed for the project, the correlation is not high and the functions that can be introduced are very limited.

2. Project 1. Acoustic Link

2.1. Stories

By referring to the formulas provided in the manual, we quickly completed the generation of sound waves. When attempting to complete the task of transmitting data, we chose PSK and used sine waves as the carrier. However, during the debugging process, we tried to adjust parameters such as the detection threshold of the preamble wave, sampling frequency, and sampling buffer, but the results were always unsatisfactory. There were situations of garbled data, and by the time the project was concluded, we still did not have a definite conclusion. Ultimately, this led to the failure of our task objective.

2.2 Designs

For the first two tasks, we completed them based on the text prompts. For tasks like generating sound waves, we adjusted the frequency of the sound waves and combined them together. For data transmission, we use a sine wave as the 0 carrier, and a phase increment of as the 1 carrier. We employ two preamble coding methods. One is a pre-defined preamble array, which is sampled and incorporated into the carrier. The receiver identifies the preamble using inner product and threshold. The other is the identification frequency of the preamble, which increases from 5000 Hz to 10000 Hz and then decreases. We use the inner product of the received samples and the preamble wave to identify the preamble. This method enables the receiver to accurately determine the starting point of the actual data transmission.

2.3. Challenges

Due to the noise issues of the audio equipment and the laptop itself, the reception effect of the audio transmitted through the air has always been poor. We have tried to disable the audio enhancement, increase the volume, and use headphones to connect to the laptop device for reception.

2.4. Solutions

Regrettably, we ultimately failed to find an effective solution for implementing the framework at that time. This prevented us from completing the task of audio signal transmis-

sion, and as a result, we were unable to accomplish the task in Project 2.

2.5. Suggestions

At the end of the project, we discovered that due to a problem with one member's equipment, the same code produced different results on different devices. When A and B were able to transmit stably without any errors, A and C were unable to receive the correct results during transmission. Therefore, we believe that the role of hardware in the success of the project might be overemphasized. Moreover, due to the high degree of interconnection between the four sub-projects, if the earlier projects fail to be completed, it will seriously affect the completion of the subsequent projects. It can be imagined that if the previous project consumed a lot of effort but was not completed, then when completing the subsequent project, the students not only need to complete the workload of the subsequent project, but also need to re-implement the previous project. The cumulative effect of the workload is significant, resulting in even though the subsequent project is not so difficult, it still requires a lot of effort to complete.

3. Project 2. Manage Multiple Access

3.1. Stories

When the transmission method changed from air transmission to wired transmission, the transmission effect improved to some extent. However, our transmission could only guarantee approximately 40% accuracy, and this accuracy still had a significant degree of fluctuation. Although we attempted to reduce the number of samples required for one bit and adjusted the sampling size to enhance stability, the results were not satisfactory. Given the failure in the implementation of the previous project, we have decided to change the implementation method and re-implement using the square wave and Line Coding techniques. However, time is not sufficient now. By the deadline, we were still in the stage of adjusting parameters and could not guarantee a very high degree of accuracy. Therefore, we did not obtain the score.

3.2. Designs

The design method has been elaborated in the "Stories" section.

3.3. Challenges

The challenges we encountered in this project are similar to those in the previous one, but there is also a problem of code refactoring. At that time, it happened to be during the mid-term period of some other courses.

3.4. Solutions

We spent a considerable amount of time on the detection and parameter adjustment of the data packets sent and received between the two devices. When we finally gave up the initial approach, it took a long time to restructure the code. Furthermore, we believe that the fact that we were not aware of the problems with the hardware equipment at

that time also caused us to expend a lot of unnecessary effort. We believe that the initial implementation method may not necessarily be ineffective. We should be able to choose other devices for testing instead.

4. Project 3. To The Internet

4.1. Stories

When completing this project, we discovered a workaround by using the built-in network packet capture tool of Python to identify and package the packet headers. This saved us from implementing this part in C++, and we only needed to add an additional layer on top of the original data transmission.

4.2. Designs

To simplify the design of the network protocol, we decided to omit the transmission of the MAC address in the audio Ethernet link layer. The audio Ethernet ICMP packet structure only contains the IP header, the ICMP header, and the corresponding data payload. The two communication nodes in the network are assigned to the same private subnet and have logical direct connectivity. We strictly followed the RFC 792 specification to construct the ICMP Echo Request and Echo Reply messages in the audio Ethernet environment. To accurately measure the round-trip time (RTT) and generate a random identification field in the ICMP message, we utilized the high-precision clock function of the chrono library to complete the corresponding timing and identification generation. In the audio Ethernet program at node 2, we enabled its wireless hotspot function and used this interface to receive and send ICMP data packets with node 3, which is connected to this hotspot. Thus, we achieved the network connectivity test across nodes.

4.3. Challenges

We encountered difficulties in optimizing the RTT time for sending and receiving packets. We suspect that it is because we used the program's data packet forwarding, which inevitably leads to performance differences. This makes it very difficult for us to improve the time without changing the structure and by merely optimizing minor details.

4.4. Solutions

We reduced the waiting time, decreased the buffer size and increased the frequency of chunk sending. However, these minor optimizations were not sufficient to enable us to reach the standard for achieving a perfect score on this project. Therefore, our first task point, ICMP Echo, did not receive a full score.

5. Project 4. Above IP

5.1. Stories

While working on this project, the Spring semester of 2024 had already begun at the University of California, Berkeley. We stayed up very late, attempting to complete op-

tional Task 5. However, due to the vague task requirements in the document, we used a completely incorrect and non-scoring method to finish Task 5, staying awake until 4 in the morning local time in vain.

5.2. Designs

At the physical layer, the JUCE audio library is used to implement 4-sample Manchester encoding. Frame synchronization is achieved through the preamble code 0x55 0x55 0x54, and data integrity of the audio transmission is ensured by CRC32 verification. Node2 acts as an intelligent gateway and realizes protocol conversion between the audio network and the IP network through a dual-port UDP Socket pipeline (9003/9004). DNS query flag filtering (checking the QR bit) is used to distinguish queries from responses, preventing audio echo interference. The Python DNS server is implemented based on Scapy to perform iterative recursive queries starting from the root server (Root → TLD → Authoritative), and extracts the A record IP through response packet reconstruction technology and re-creates a standardized DNS response packet to ensure compatibility with the simple analyzer of Node1.

Node1 acts as the client and encapsulates HTTP requests using a 4-sample Manchester encoded audio TCP protocol stack. The main audio channel transmits data frames that have undergone CRC verification. At the same time, a UDP port 1234 is established as a backup channel to ensure transmission fault tolerance. Node2 acts as an intelligent protocol gateway. Firstly, it parses the audio TCP packet and verifies the specific sequence number. Then, it uses JUCE StreamingSocket to establish a connection to the real web server and writes the HTTP response fragments into the local index.html file. After completion, it calls the default browser through the system command for rendering. It also inherits the input URL input box for the user interface and has an internal domain-IP mapping table to handle DNS resolution.

5.3. Challenges

This project actually adds the protocol conversion and recursive query layer on the basis of Project 3. It integrates the complete TCP/IP protocol stack, application layer processing, file system integration, and browser invocation layer. For Task 1, the DNS protocol resolver, port 53 filtering mechanism, Python recursive parsing service and response re-configuration mechanism are relatively complicated. For Task 2, we need to simulate TCP connections at the audio layer, support the SYN/ACK/PSH/FIN flags and sequence number management, support HTTP request construction and response parsing, and establish a real TCP connection to the Internet server. This is rather troublesome. Task 5, I think how to capture the browser page and save it in time for rendering is quite difficult. Here, we take an expedient approach by integrating a domain-to-IP conversion table and bypassing real-time DNS resolution.

5.4. Solutions

The solution has been described in the "Designs" section.

6. Overall Suggestions

On one hand, it is hoped that the guidance instructions in the task documents can be expressed more clearly and specifically, so as to prevent learners from wasting a lot of time on the wrong path due to misunderstandings. On the other hand, it is suggested that the course should appropriately reduce its reliance on specialized audio network experimental equipment, because its high hardware requirements have inadvertently increased the economic burden on students and also easily introduced some difficult-to-diagnose software and hardware compatibility issues. This has caused significant problems in Project 1 and Project 2.